

Domain Research → Classes, Subclasses, and Patterns

A repeatable process for turning requirements or observational research (e.g., studying a real system) into a class model — deciding when a class should split into pieces, when it needs a design pattern, and how to wire the resulting classes together with the right relationships.

Step 1: Turn the research into nouns and verbs first

Read through requirements/observations and extract two plain lists: things (nouns) and actions (verbs). Don't judge them yet.

Example (from FGCU bookstore research): nouns like *cart*, *return*, *order*, *course*; verbs like *request*, *approve*, *search*, *download*.

Step 2: For each noun — is it a class, or just a property of one?

Test: does it have its own identity and data that outlives a single use, or is it just describing something else?

- Book — has ISBN, title, price; exists independently; gets referenced by many other things → **class**.
- “Price range” — only exists as a filter someone types into a search box, has no identity of its own → **not a class**, just a parameter (this is why it belongs as a field on SearchCriteria, not as its own PriceRange class).

If you can't picture someone asking “give me *that specific one*” about it, it's probably an attribute, not a class.

Step 3: For each class — does it need subclasses?

Test: do these things share a role but *behave differently* when that role is exercised — not just have different data?

- PhysicalBook / EBook — pass this test: both are “a Book you can buy,” but *how* you fulfill them genuinely differs (ship vs. download).
 - Counter-example: if “hardcover” and “paperback” only differed by a format: string field and nothing about their *behavior* changed, that's an attribute, not a subclass. Making it a subclass anyway is over-modeling — the same mistake as duplicating Student/Faculty as identical subclasses of User with nothing distinguishing them.
-

Step 4: For each class doing multiple things — should it split?

First ask the question that actually decides it: are the parts *sequential stages of one lifecycle*, or *unrelated responsibilities* that only coincidentally share an actor or a piece of data?

- **Sequential stages → keep it as one class, and use State** (Step 5). A different actor triggering each stage does **not** by itself mean split — a request moving from “submitted” to “decided” is one continuous process even when two different people carry it through those stages. (*ReturnRequest and BorrowRequest — request → decision is one lifecycle, even though a customer/member triggers the first stage and staff triggers the second. One class, State pattern, not two classes.*)
- **Unrelated responsibilities → split.** No sequence connects them; they just happen to live on the same class. (*Admin originally had addBook()/removeBook() **and** approveReturn() directly on it — same single actor the whole time, so “different actor” was never even in play here. What made this a real split: managing the catalog and approving a return aren’t stages of one process — nothing about finishing one leads into starting the other.*)

If you’re still unsure after that question, run these four checks. If **any** come back “yes, they differ,” split the class:

1. **Reason to change?** If the business changed the *rule* for one part, would you have to touch code for the other, unrelated part? If not, they don’t belong together. This is SRP’s actual test — not “how many methods does the class have.”
2. **Rate of change?** Does one part change constantly while the other barely changes? (*A stock count updates on every sale; a book’s title almost never changes — a real signal they’re different responsibilities even if both are “about the same book.”*)
3. **Timing/lifecycle gap — but check it’s a genuine gap, not just a handoff.** Does one part happen, then the class sits idle for an unrelated reason, then something unconnected happens to it later? Don’t confuse this with a request simply waiting for review — that waiting is a stage of the same lifecycle, not a gap between two different ones.
4. **Do the “parts” actually depend on each other’s data?** If approving requires reading what was requested, that’s one object referencing its own history — not two objects that happen to relate.

If **none** of these fire, keep it as one class. Don’t split something that belongs together just to look thorough — that’s the opposite failure mode (KISS), same lesson as only splitting CatalogService/ReturnService apart because they genuinely had different callers, not by default.

Step 5: For each remaining behavior — does it match a pattern signal?

Use a pattern signal table (e.g., the GoF quick reference), but apply it **after** steps 1–4, not instead of them. A pattern is the answer to “this specific relationship has this specific shape” — it is never the starting point. Avoid signal-hunting for a pattern before confirming the class itself is real and correctly scoped.

One more distinction worth knowing before checking a signal table. Creational and Behavioral patterns are almost always about a decision or event happening while the program runs — the test is usually “*does this decision get made repeatedly, at runtime, from more than one place?*” (this is exactly the test that separates “needs subclasses” from “needs Factory Method” — subclasses answer

whether things are structurally different; Factory Method answers whether *choosing between them* is a repeated runtime operation). Structural patterns are different — they're often a fixed relationship between classes that's simply true once designed that way, not something that "happens" at runtime (a Composite tree has its shape whether or not the program is currently running). Don't apply the runtime-repetition test to a Structural pattern; it's the wrong question for that category.

Step 6: For every pair of related classes — what kind of line?

Once classes, subclasses, and patterns are settled, wire them together. Generalization (inheritance) and realization (interface implementation) are usually already decided by Steps 3 and 5. Everything else — every remaining line on the diagram — comes down to one decision cascade, run per pair:

1. **Is this already an is-a relationship?** → Generalization. (Already handled in Step 3.)
2. **Does A only touch B briefly — inside one method, as a parameter or return value — and never store it?** → **Dependency** (dashed arrow). This is the right call for most Builder/Factory relationships: a PaymentFactory creates a Payment Method and hands it off; it doesn't keep one.
3. **Does A store a permanent reference to B, but B has its own independent existence and could outlive A or belong to something else?** → **Association** (plain solid line). Most relationships end up here — a Cart referencing Books that exist in the catalog regardless of any one cart, for example.
4. **Does A permanently own B, such that B has no meaning and no reason to exist without A — if A is destroyed, B goes with it?** → **Composition** (solid line, filled diamond at the "whole"). This is the right call for a State-pattern's current-state field (Order's specific OrderState object only exists to describe that order) and for ownership relationships with no independent lifecycle (a Customer's Cart, if the design has no concept of a cart existing without an owner).
5. **For every line, label both ends with multiplicity** (1, 0..1, 0..*, 1..*). Ask, in both directions: *for one A, how many B's — and for one B, how many A's?* "One order, one payment method" and "one order, many payment methods" are different design claims, not an interchangeable pair — get this from what you actually observed/decided, not from habit.

Aggregation (hollow diamond) exists between association and composition, but the distinction from plain association is notoriously blurry even in professional diagrams. Default to association unless composition's test clearly fires — adding aggregation diamonds everywhere "to show ownership" is clutter without added meaning, the same KISS instinct from Step 4.

A pattern's own definition often answers the question for you. If a relationship is the connection *between* a class and the pattern support classes it uses (e.g., Order and its OrderState family, or OrderCheckoutSession and the Order it builds), re-read what that pattern is actually supposed to do — State's whole point is an owned, disposable state object (composition); Builder's whole point is momentary assembly, not permanent ownership (dependency). Don't re-derive from scratch what the pattern already tells you.

The meta-skill: recognize a shape you've already solved

This is the real payoff of working through an example like Return carefully. “Different actor requests vs. approves, and the thing moves through named stages” is *one shape*. Once you've solved it correctly (Return → State), you should be able to recognize the *identical* shape elsewhere (Order Status → State) without being told twice.

The actual skill domain research builds isn't memorizing pattern names — it's noticing “I've seen this exact structure before” and reaching for the same solution. A signal table is a lookup aid for when you're rusty; the recognition itself is what transfers to a different bookstore, or to a system that isn't a bookstore at all.